Zen Training Methodology: From Pretraining to Deployment

Technical Report v2025.03

Antje Worring, Zach Kelling, Zen LM Research Team
research@zenlm.org

March 2025

Abstract

We present the complete training methodology for the Zen family of large language models, spanning pretraining on 7 trillion tokens through supervised fine-tuning and reinforcement learning from human feedback. Our approach introduces several key innovations: a hybrid data mixing strategy that balances quality and diversity, perplexity-based quality filtering that removes low-signal content, and a constitutional training regime that instills safety properties without sacrificing capability. We report detailed training curves, compute budget analysis, and ablation results demonstrating the contribution of each methodological component. Models trained under this methodology achieve state-of-the-art performance on standard benchmarks while maintaining robust safety properties and deployment reliability. The Zen MoDE (Mixture of Distilled Experts) architecture enables efficient scaling from 600M to 480B parameters under a unified training pipeline.

1 Introduction

Training frontier language models requires careful coordination of data curation, architectural decisions, and optimization strategies across a multi-stage pipeline. The Zen training methodology addresses the full lifecycle: from raw web data ingestion through pretraining, instruction tuning, and alignment, to final deployment optimization.

Key challenges we address include:

- **Data quality at scale**: Curating 7T tokens that balance breadth and quality
- **Stable large-scale optimization**: Preventing loss spikes and ensuring smooth convergence
- **Alignment without degradation**: Instilling safety properties while preserving capability
- **Compute efficiency**: Achieving optimal FLOPs allocation across model sizes

The Zen MoDE architecture introduces a Mixture of Distilled Experts paradigm, where each expert specializes in semantic domains while sharing a unified vocabulary and positional encoding scheme. This architectural choice significantly impacts training dynamics and data mixing decisions.

2 Background and Related Work

2.1 Scaling Laws

The Chinchilla scaling laws [1] established that optimal training requires roughly 20 tokens per parameter. Subsequent work has shown that inference-optimal training often favors over-training

smaller models on more tokens. The Zen family follows an inference-optimal strategy, training smaller models on significantly more data than Chinchilla-optimal prescriptions.

2.2 Data Curation

Prior work on data curation has emphasized the importance of deduplication, quality filtering, and domain mixing. C4 [2], The Pile [3], and RedPajama [4] established key preprocessing pipelines. Zen extends these with perplexity-based filtering and domain-aware mixing.

2.3 Multi-Stage Training

The standard paradigm of pretraining followed by instruction tuning (SFT) and RLHF was established by InstructGPT [5]. Zen introduces constitutional training as an intermediate stage between SFT and RLHF, reducing human annotation requirements while improving alignment quality.

3 Data Curation Pipeline

3.1 Raw Data Collection

The Zen pretraining corpus aggregates data from multiple sources:

Source	**Raw Size**	**After Filter**	**Mix Weight**
Web (Common Crawl)	68T tokens	4.2T	60%
Books & Literature	400B tokens	380B	12%
Scientific Papers	150B tokens	140B	8%
Code Repositories	800B tokens	650B	12%
Wikipedia + Encyclopedias	80B tokens	78B	3%
Curated Q&A	120B tokens	110B	5%
Total	**∼70T**	**7T**	**100%**

Table 1: Zen pretraining corpus composition after filtering.

3.2 Quality Filtering

Our quality pipeline applies filters in sequence, each removing a fraction of documents:

Algorithm 1 Zen Quality Filtering Pipeline

Require: Raw document corpus $\mathcal{D}$
Ensure: Filtered corpus $\mathcal{D}^*$

```
D_1 ← LanguageID(D), keep p(en) > 0.65
D_2 ← URLFilter(D_1), remove spam/adult domains
D_3 ← MinHashDedup(D_2, ngram = 13, jaccard > 0.8)
D_4 ← HeuristicFilter(D_3):
    Remove docs < 100 tokens or > 100K tokens
    Remove docs with symbol/word ratio > 0.1
    Remove docs with repeated n-gram ratio > 0.3
Train reference LM M_ref on high-quality seed corpus (200B tokens)
D_5 ← {d ∈ D_4 : PPL_{M_ref}(d) < τ_ppl}
τ_ppl ← 90th percentile of PPL_{M_ref} on D_4
D* ← DomainSample(D_5, weights = W)
return D*
```

3.2.1 Perplexity-Based Quality Filtering

A key innovation is using a reference language model to score document quality. We train a 1.3B parameter reference model on a manually curated seed corpus of 200B high-quality tokens. Documents scoring above the 90th percentile perplexity threshold are removed, eliminating repetitive, incoherent, or template-generated content.

The perplexity filter reduces web content from 68T to 4.2T tokens (6.2% retention), while books and scientific papers show 95%+ retention, confirming the filter's discriminative power.

3.3 Hybrid Data Mixing

Static mixing ratios degrade performance as training progresses because the model learns different domains at different rates. We employ adaptive mixing via a domain loss monitor:

$$w_d^{(t+1)} = w_d^{(t)} \cdot \exp\left(\eta \cdot \frac{\mathcal{L}_d^{(t)} - \bar{\mathcal{L}}^{(t)}}{\sigma_{\mathcal{L}}^{(t)}}\right) \tag{1}$$

where $w_d^{(t)}$ is the weight for domain d at step t, $\mathcal{L}_d^{(t)}$ is the per-domain validation loss, and $\eta = 0.01$ is the adaptation rate. Weights are normalized after each update. This causes the optimizer to allocate more capacity to domains where the model lags.

4 Pretraining Setup

4.1 Architecture: Zen MoDE

The Zen MoDE (Mixture of Distilled Experts) architecture replaces dense FFN layers with a mixture of E expert networks, activated sparsely:

$$\text{MoDE}(x) = \sum_{i=1}^{k} g_i(x) \cdot E_i(x), \quad g(x) = \text{TopK}(\text{softmax}(W_g x), k) \tag{2}$$

where $k = 2$ experts are selected per token from $E = 64$ total experts. Expert specialization emerges from domain-structured data: we observe that distinct experts activate preferentially for code, mathematics, and natural language.

4.2 Model Configurations

Model	Params	Layers	d_model	Heads	Experts
Zen-600M	600M	24	1024	16	–
Zen-7B	7B	32	4096	32	–
Zen-32B	32B	64	7168	56	–
Zen-235B-MoE	235B	94	7168	56	128
Zen-480B-MoE	480B	128	8192	64	256

Table 2: Zen model family configurations.

4.3 Optimization

All models are trained with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay $\lambda = 0.1$). Learning rate follows a warmup-cosine schedule:

$$\eta_t = \eta_{\max} \cdot \begin{cases} t/T_{\text{warm}} & t < T_{\text{warm}} \\ \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{t - T_{\text{warm}}}{T - T_{\text{warm}}}\right)\right) & t \geq T_{\text{warm}} \end{cases} \tag{3}$$

with $T_{\text{warm}} = 2000$ steps, $\eta_{\max}$ model-size dependent (see Table 3), and final learning rate $\eta_{\min} = \eta_{\max}/10$.

Model	Peak LR	Batch Size (tokens)	Train Steps
Zen-600M	6×10^{-4}	2M	3.5M
Zen-7B	3×10^{-4}	4M	1.75M
Zen-32B	1×10^{-4}	8M	875K
Zen-235B-MoE	5×10^{-5}	16M	437K
Zen-480B-MoE	3×10^{-5}	32M	219K

Table 3: Training hyperparameters per model size.

4.4 Compute Infrastructure

Training runs on H100 SXM5 80GB clusters with NVLink interconnects. Pipeline parallelism depth P, tensor parallelism T, and data parallelism D are configured per model size:

Model	GPUs	P	T	D
Zen-7B	512	1	4	128
Zen-32B	1024	4	8	32
Zen-235B-MoE	2048	8	8	32
Zen-480B-MoE	4096	16	8	32

Table 4: Parallelism strategies per model. MoE models use expert parallelism with EP=64.

5 Supervised Fine-Tuning

5.1 SFT Data Construction

The SFT dataset contains 2.4M instruction-response pairs spanning: multiturn conversation (35%), coding tasks (25%), mathematical reasoning (20%), long-form writing (12%), and factual Q&A (8%). All pairs are human-validated or model-generated with human verification.

5.2 SFT Training Protocol

SFT is conducted for 3 epochs with learning rate 5×10^{-6}, batch size 256, and maximum sequence length 32K tokens. Only response tokens contribute to the cross-entropy loss:

$$\mathcal{L}_{\text{SFT}} = -\sum_{t \in \text{response}} \log p_\theta(x_t \mid x_{<t}) \tag{4}$$

We apply NEFTune noise injection [6] with $\alpha = 5$ to improve generalization on open-ended tasks.

6 Constitutional Training

Between SFT and RLHF, we apply a constitutional training stage that teaches the model to critique and revise its own outputs according to a set of principles:

1. Generate response r_0 to prompt p
2. Apply critique template: "Identify ways the response violates principle c_i"
3. Generate revision r_1 guided by the critique
4. Train on (p, r_1) pairs using SFT loss

This reduces human annotation requirements for the subsequent RLHF stage by 40% while improving harmlessness scores by 12 points (see Section 7).

7 RLHF Pipeline

7.1 Reward Model Training

The reward model (RM) is initialized from the Zen-7B SFT checkpoint and trained on 400K human preference pairs. Given two responses r_a, r_b to the same prompt, the RM is trained to maximize the margin:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(p,r_w,r_l)} \left[\log \sigma \left(r_\phi(p, r_w) - r_\phi(p, r_l)\right)\right] \tag{5}$$

where r_w is the preferred response and r_l the rejected response.

7.2 PPO Training

Policy optimization uses Proximal Policy Optimization with the KL penalty:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\left[r_\phi(p, r) - \beta \cdot \text{KL}\left[\pi_\theta(\cdot|p) \| \pi_{\text{ref}}(\cdot|p)\right]\right] \tag{6}$$

$\beta = 0.04$ is tuned to balance helpfulness and proximity to the SFT policy. We apply a reward clipping of $[-5, 5]$ and run 2 PPO epochs per batch.

8 Experiments and Results

8.1 Training Curves

Pretraining loss curves exhibit three distinct phases observed across all model sizes: (1) rapid descent in the first 5% of training as the model learns basic token statistics, (2) steady decline through 90% of training as domain knowledge accumulates, (3) a slower final phase with diminishing returns signaling data saturation.

Loss spike prevention is achieved via gradient norm clipping at 1.0 and a loss spike detector that halves the learning rate for 100 steps when $\mathcal{L}_t > 1.5 \cdot \bar{\mathcal{L}}_{t-100:t}$.

8.2 Ablation Results

Configuration	MMLU	HumanEval	MATH	MT-Bench
Full pipeline	**85.3**	**78.2**	**67.4**	**8.62**
No PPL filter	83.1	76.4	64.8	8.31
Static mix (no adaptive)	84.0	77.1	65.9	8.44
No constitutional training	84.9	78.0	67.1	8.41
No NEFTune	84.8	77.8	67.2	8.39

Table 5: Ablation study on Zen-7B. All numbers are pass@1 or accuracy (%).

8.3 Compute Budget Analysis

We analyze the FLOPs-to-performance tradeoff by training model variants with different compute budgets:

$$\text{Performance}(C) \approx A - B \cdot C^{-\alpha} \tag{7}$$

where C is the training compute in FLOPs, and we fit $A = 89.1$, $B = 142.3$, $\alpha = 0.095$ for MMLU performance of Zen-7B. This implies diminishing returns beyond 3×10^{23} FLOPs for this model size, motivating the switch to larger models for higher capability targets.

9 Analysis

9.1 Data Quality vs. Quantity

Our experiments confirm that data quality dominates quantity beyond a threshold. Doubling corpus size with the PPL filter disabled yields +0.8 MMLU points, while halving corpus size with tighter filtering (τ_{ppl} at 80th percentile) yields +1.1 MMLU points.

9.2 Expert Specialization in MoDE

Analysis of expert activation patterns in Zen-235B-MoE reveals clear domain specialization: code-related tokens activate a consistent subset of 8–12 experts, mathematics activates a partially overlapping set, and natural language distributes more broadly. This specialization emerges organically from the data without explicit routing supervision.

9.3 Scaling Behavior

Zen models follow a modified scaling law that accounts for MoE efficiency:

$$\mathcal{L}(N_{\text{active}}, D) = \frac{A}{N_{\text{active}}^{\alpha}} + \frac{B}{D^{\beta}} \tag{8}$$

where N_{active} is the number of active parameters per token (not total parameters), D is the dataset size, $\alpha = 0.076$, $\beta = 0.095$. MoE models achieve lower loss at fixed active-parameter compute compared to dense models.

10 Conclusion

The Zen training methodology demonstrates that careful data curation, adaptive mixing, constitutional training, and staged alignment produce models that are both capable and safe. The perplexity-based filtering provides the largest single capability gain among the methodological components we study. Constitutional training between SFT and RLHF reduces annotation costs while improving alignment quality, making the pipeline more scalable.

Future work will explore continual pretraining strategies, domain-specific fine-tuning at scale, and automated data quality assessment to further reduce human annotation requirements in the training pipeline.

References

[1] Hoffmann et al. (2022). Training Compute-Optimal Large Language Models. *NeurIPS*.

[2] Raffel et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*.

[3] Gao et al. (2021). The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *arXiv:2101.00027*.

[4] Together AI (2023). RedPajama: An Open Source Recipe to Reproduce LLaMA Training Dataset. *GitHub*.

[5] Ouyang et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*.

[6] Jain et al. (2023). NEFTune: Noisy Embeddings Improve Instruction Finetuning. *arXiv:2310.05914*.